

Word Equations outcome = explanatory + other stuff Y = X + other stuff	Summary Tables <pre># compute five-number summary favstats(~ Y, data = data_set) # create frequency table tally(data_set\$Y) tally(~ Y, data = data_set) # tally by condition tally(~ Y < 1900, data = data_set) # two-way frequency table tally(Y ~ X, data = data_set, margin = TRUE, format = "proportion")</pre>	Simple Statistics <pre>mean(data_set\$Y) var(data_set\$Y) sd(data_set\$Y) cohensD(Y ~ X, data = data_set) cor(Y ~ X, data = data_set) b1(Y ~ X, data = data_set) b1(my_model) pre(Y ~ X, data = data_set) f(Y ~ X, data = data_set)</pre>
Basics <pre>print("Hello world!") # assign value to object my_number <- 5 # combine values into vector my_vector <- c(1, 2, 3) # first element in vector my_vector[1] # orders values or cases sort(my_vector) # arithmetic operations sum(1, 2, 100), +, -, *, / sqrt(157) abs(data_set\$Y) # logical operations >, <, >=, <=, ==, !=, , & # results in a new variable with values # of TRUE or FALSE data_set\$C <- data_set\$A > data_set\$B</pre>	Data Frame <pre># structure of data frame str(data_set) # view first/last six rows head(data_set) tail(data_set) # select multiple variables select(data_set, Y1, Y2) # first six rows of selected variables head(select(data_set, Y1, Y2)) # select variable (a column) data_set\$Y # find rows that meet some condition data_set[data_set\$Y > 40] filter(data_set, Y > 300) # find rows that do not have NA filter(data_set, is.na(Y) == FALSE) filter(data_set, !is.na(Y))</pre>	<pre># arrange rows by variable arrange(data_set, Y) # creates data frame from csv file data_set <- read.csv("file_name", header = TRUE) # convert quantitative variable # to categorical factor(data_set\$Y) factor(data_set\$Y, levels = c(1,2), labels = c("A", "B")) # transform values recode(data_set\$Y, "0" = 0, "1" = 50, "2" = 100) # creates two equal sized groups ntile(data_set\$Y, 2) # convert categorical variable # to quantitative as.numeric(data_set\$Y)</pre>
Probability Distribution <pre># calculate the probability area xpnorm(65.1, data_set\$mean, data_set\$sd) zscore(data_set\$Y) # returns t at this probability qt(.975, df = 999) # returns F at this probability qf(.95, df1 = 1, df2 = 100) # CI using t distribution confint(empty_model) # calculate p-value using F-distribution xpf(sample_f, df1 = 2, df2 = 10)</pre>		

Simulation

```
# sample without replacement
sample(data_set, 6)

# sample with replacement
resample(data_set, 10)

do(3) * resample (data_set, 10)

# mixes up values in a variable
shuffle(data_set$Y)

# simulate sampling 10000 Ys
# from normal distribution
sim_Y <- rnorm(10000, Y_stats$mean,
Y_stats$sd)

# put simulated Ys into dataframe
data_set<- data.frame(sim_Y)

# simulate
# sampling distribution of means
sdom_sim <- do(10000) * mean(rnorm(157,
Y_stats$mean, Y_stats$sd))

# bootstrap
# sampling distribution of means
sdom_boot <- do(10000) *
mean(resample(data_set$Y, 157))

# randomize sampling distribution of bls,
# centered on 0
sdo_b1 <- do(1000) *
  b1(shuffle(Y) ~ X, data = data_set)

# bootstrap sampling distribution of bls,
# centered on sample b1
sdo_b1_boot <- do(1000) *
  b1(Y ~ X, data = resample(data_set))

# count the number of bls at the upper
# and lower extreme
tally(sdo_b1$b1 > sample_b1 |
sdo_b1$b1 < -sample_b1)

# return TRUE for middle 95% of distribution
middle(sdo_b1$b1, .95)

# randomize sampling distribution of PREs
sdopre <- do(1000) * pre(shuffle(Y) ~ X,
data = data_set)

# randomize sampling distribution of Fs
sdof <- do(1000) *
  f(shuffle(Y) ~ X, data = data_set)

# counts extreme Fs
sample_f <- f(shuffle(Y) ~ X, data = data_set)
tally(~f > sample_f, data = sdof)
```

Fitting and Evaluating Models

```
# empty model
empty_model <- lm(Y ~ NULL, data = data_set)

# use one explanatory variable
my_model <- lm(Y ~ X, data = data_set)

# model predictions and residuals
data_set$empty_predict <- predict(empty_model)
data_set$empty_resid <- resid(empty_model)

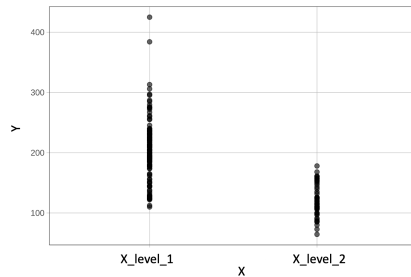
# produce ANOVA table
anova(empty_model)
supernova(my_model)

# t-test, using pooled variance
t.test(Tip ~ Condition, data = data_set,
var.equal=TRUE)

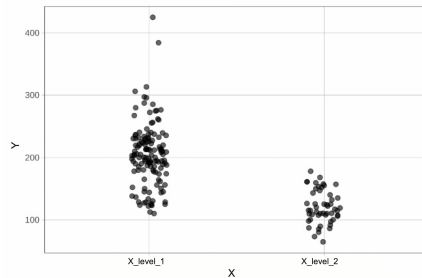
# pairwise comparison
# corrections: "Tukey", "Bonferroni", "none"
pairwise(my_model, correction = "Tukey")
```

Visualizations

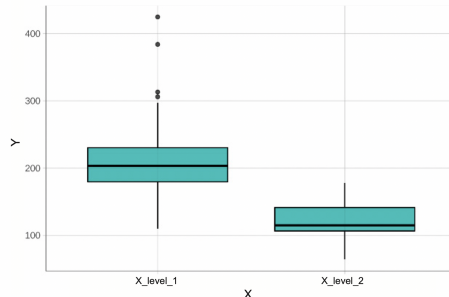
```
gf_point(Y ~ X, data = data_set)
```



```
gf_jitter(Y ~ X, data = data_set)
```

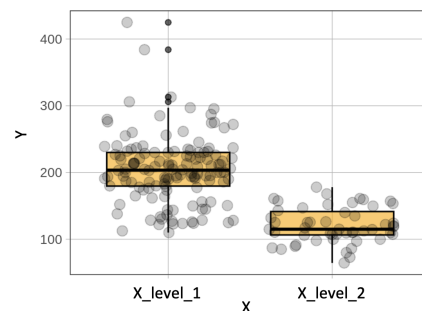


```
gf_boxplot(Y ~ X, data = data_set)
```

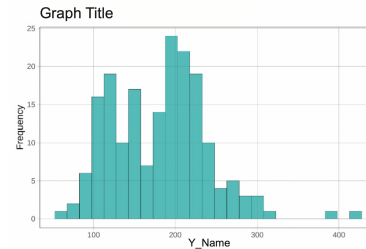


```
gf_boxplot(Y ~ X, data = data_set, fill = "orange") %>%
```

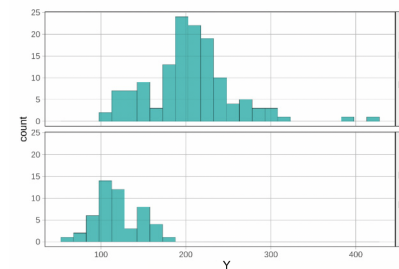
```
gf_jitter(height = 0, alpha = .2, size = 3)
```



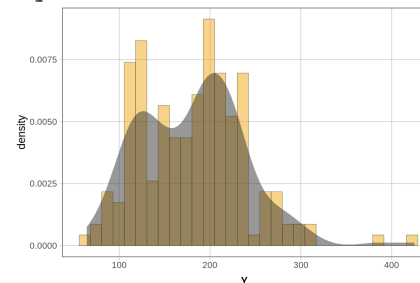
```
gf_histogram(~ Y, data = data_set) %>%
# change labels
gf_labs(title = "Graph Title",
x = "Y_Name", y = "Frequency")
```



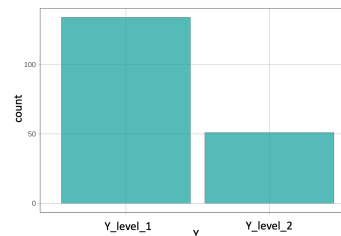
```
# faceted grid of histograms
gf_histogram(~ Y, data = data_set) %>%
gf_facet_grid(X ~ .)
```



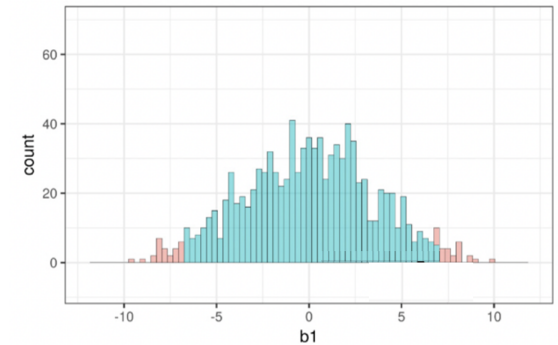
```
gf_dhistogram(~ Y, data = data_set,
fill = "orange") %>%
gf_density()
```



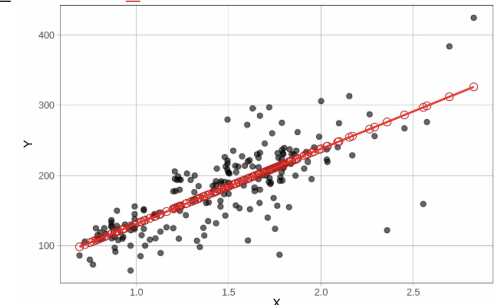
```
gf_bar(~ Y, data = data_set)
```



```
# sampling distribution of b1
gf_histogram(~b1, data = sdob1,
fill = ~middle(b1, .95)) %>%
# modify the limits on x- and y-axes
gf_lims(x = c(-12, 12), y = c(0, 70))
```



```
gf_point(Y ~ X, data = data_set) %>%
# add model predictions as red points
gf_point(Y ~ X, shape = 1, size = 3,
color = "firebrick") %>%
# add best fitting model as a red line
gf_model(my_model, color = "red")
```



```
pairwise(my_model, plot = TRUE)
```

